SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
CASE STUDY

Code of Conduct:

I _____Kanmani L/192511414_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Weak Password Hashing in Web Application
A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.
Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

---

2. Digital Signature Compromise in Financial System
A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.
Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

---

3. Kerberos Authentication Failure in Distributed System
A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.
Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

---

4. Certificate Authority Breach in PKI System
A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.
Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

---

5. Integrity Verification in Cloud Data Storage
A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.
Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. **Weak Password Hashing in Web Application**

   A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables. Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

Answer:

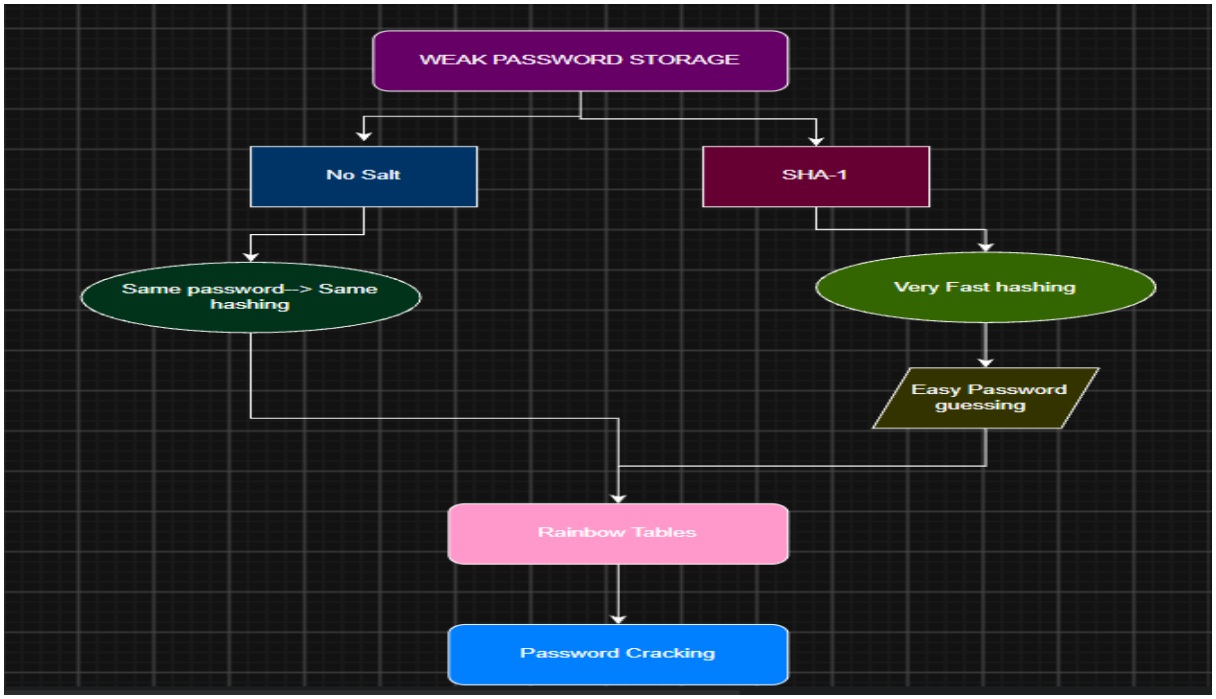
Password hashing converts a password into a one-way hash before storing it. However, SHA-1 is a fast general-purpose hash, so it is not suitable for password storage. A salt is a random value added to each password before hashing.

Analysis

- The web application stores passwords using unsalted SHA-1 hashes. This creates two major security problems.
- First, because there is no salt, users with the same password will have the same hash. Attackers can use precomputed rainbow tables containing common passwords and their hashes to quickly identify passwords.
- Second, SHA-1 is designed to be computationally fast. Although SHA-1 has known collision weaknesses, the more important issue for password storage is that its speed allows attackers to perform a very large number of password guesses.

For example:

- password123 → SHA-1 → 482c811da5...
- password123 → SHA-1 → 482c811da5...



An attacker can recognize that the same password is used by multiple accounts.

Recommended Solution;

A password-specific algorithm such as Argon2id should be used with a unique random salt for every password.

The recommended process is:

Password + Random Salt
↓
Argon2id
↓
Stored Hash + Salt

Argon2id is designed to make password guessing expensive in terms of time and memory, making large-scale attacks more difficult.

If Argon2id is unavailable, bcrypt or PBKDF2 can also be used with appropriate parameters.

Conclusion:

The use of unsalted SHA-1 made the application vulnerable to rainbow tables and rapid password guessing. Replacing SHA-1 with Argon2id and a unique random salt provides much stronger protection for stored password.

2. **Digital Signature Compromise in Financial System**

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

Answer:

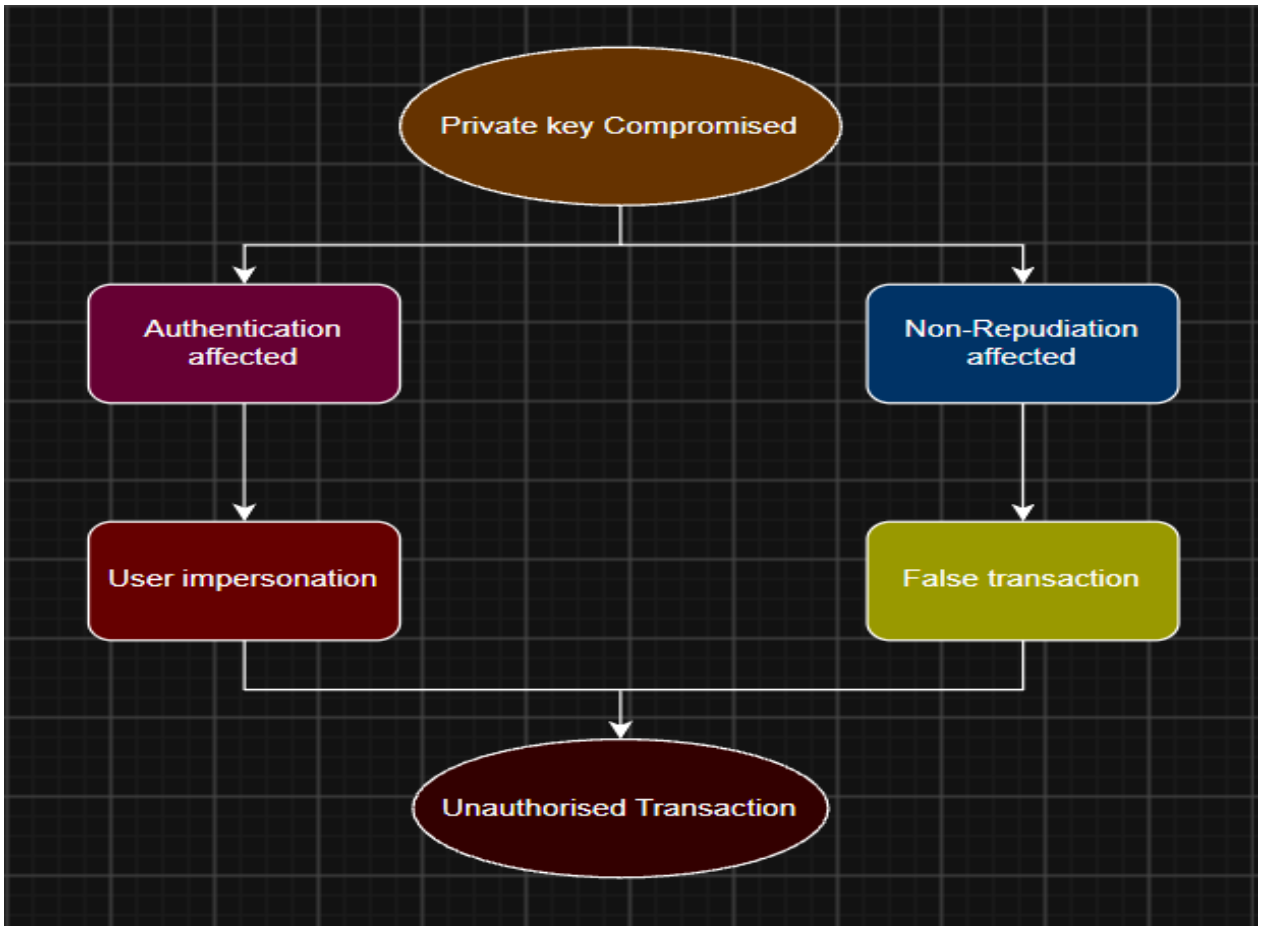
A digital signature is created using a private key and verified using the corresponding public key. It provides authentication, integrity, and non-repudiation.

Analysis:

In this case, the attacker gains access to a user's private key. This is a serious security problem because anyone possessing the private key can generate valid digital signatures.

The attacker can therefore create a fraudulent transaction and sign it with the compromised private key.

Attacker obtains Private Key
↓
Creates Digital Signature
↓
Fake Transaction
↓
Valid Signature
↓
Unauthorized Access



Mitigation Strategies:

1. Secure key storage: Store private keys in an HSM or secure hardware so they cannot easily be extracted.
2. Multi-factor authentication: Require an additional authentication factor for sensitive transactions.
3. Transaction signing: Include transaction details in the signed data so a signature cannot be reused for another transaction.
4. Key rotation: Replace keys periodically according to the organization's security policy.
5. Certificate revocation: Immediately revoke the certificate associated with a compromised key.
6. Transaction monitoring: Detect unusual transactions and require additional verification.

A compromised private key can allow attackers to impersonate users and authorize fraudulent transactions. Secure hardware storage, MFA, transaction monitoring, and immediate key revocation are essential for protecting financial systems

3.**Kerberos Authentication Failure in Distributed System**

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

Answer: Kerberos is a network authentication protocol that uses tickets and timestamps to securely authenticate users and services. It relies heavily on synchronized clocks to prevent replay attacks.

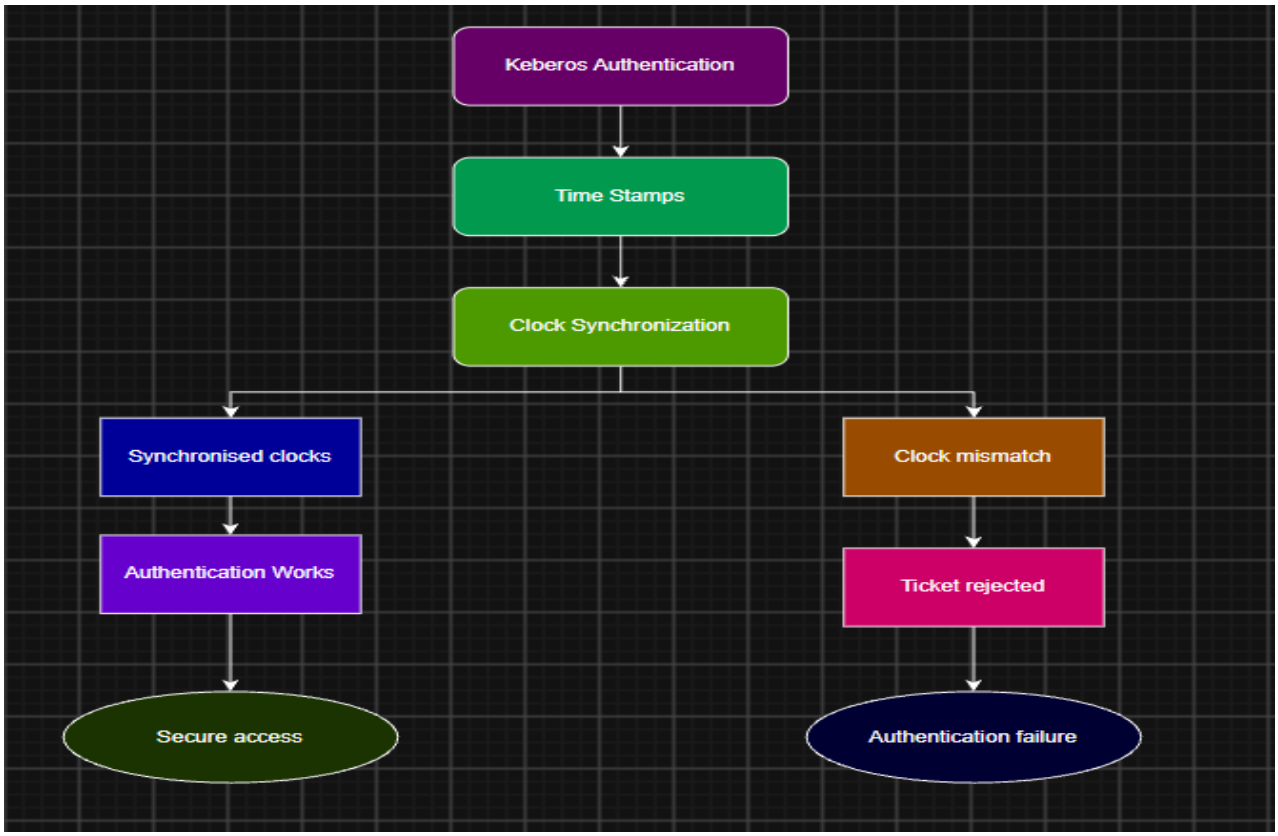
- Kerberos tickets contain timestamps that indicate when they were issued and how long they are valid. The receiving server checks the timestamp against its own system time.
- If the client and server clocks are significantly different, a valid authentication request may be rejected.

Client Time: 10:00 AM

Server Time: 10:10 AM
↓
Large Time Difference
↓
Kerberos Ticket Rejected
↓
Authentication Failure

Time synchronization is therefore important for both authentication reliability and security.

If timestamps were not checked properly, attackers could potentially replay previously captured authentication messages.



Solutions:

1. NTP:Use Network Time Protocol (NTP) to synchronize clocks across clients and servers.

2. Reliable Time Source:Configure systems to use trusted and reliable time servers.

3. Monitor Clock Drift:Continuously monitor systems for significant differences in system time.

4. Configure Clock-Skew Tolerance:Kerberos can allow a small permitted time difference between systems. This should be configured appropriately rather than using an unnecessarily large tolerance.

5. Centralized Time Management:Use consistent time synchronization policies across all servers and clients.

Using reliable NTP services, monitoring clock drift, and maintaining an appropriate clock-skew setting can ensure reliable and secure authentication.

4.Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

Answer: A Certificate Authority (CA) is a trusted entity that issues digital certificates. These certificates bind a domain or identity to a public key. Browsers and other systems trust certificates because they ultimately chain back to trusted CA certificates.

Analysis:

If a CA is compromised, an attacker may obtain the ability to issue fraudulent certificates for trusted domains.

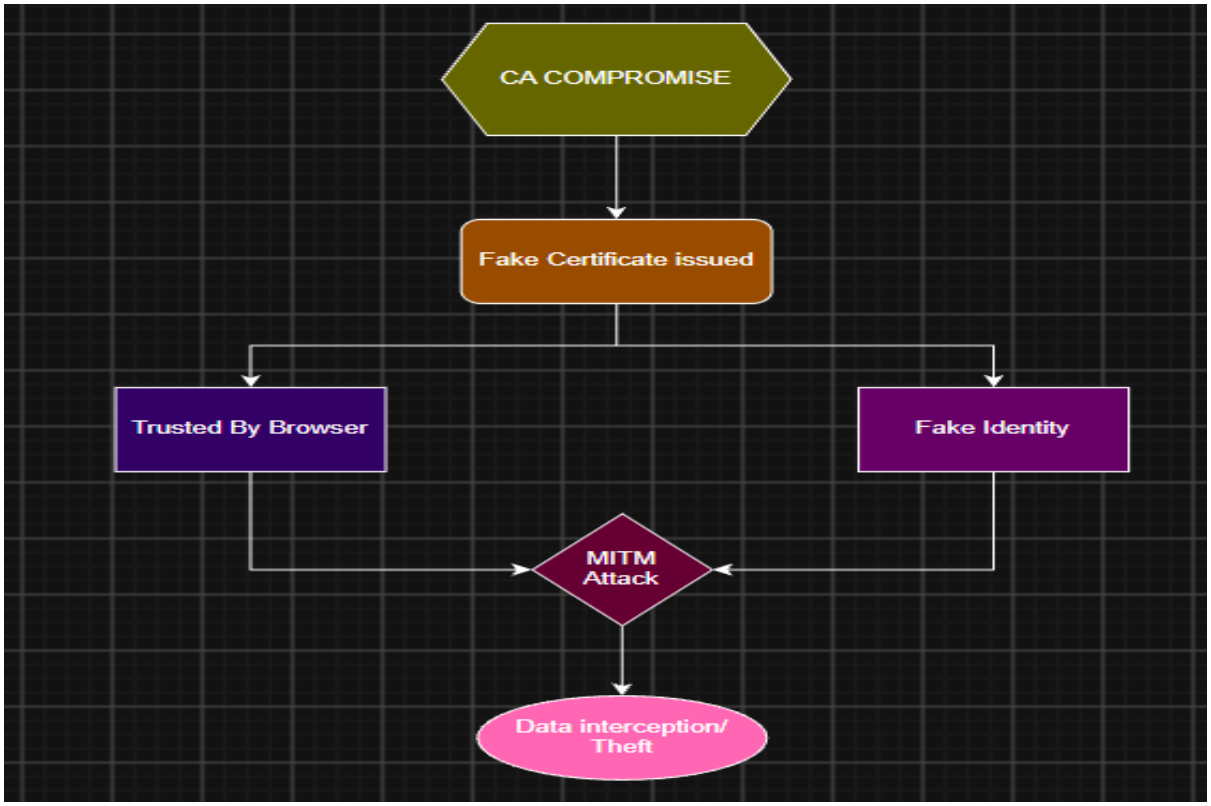
For example:

Legitimate CA
↓
Attacker obtains CA access
↓
Fake certificate for bank.com
↓
Browser trusts certificate
↓
Attacker impersonates bank.com

This can enable Man-in-the-Middle (MITM) attacks. Users may believe they are communicating securely with a legitimate website while actually communicating with an attacker.

Impact:

- Loss of trust in the compromised CA.
- Fake certificates can be issued for trusted domains.
- Attackers may impersonate websites.
- Encrypted communication may be intercepted.
- Confidential information such as passwords or financial data may be exposed.



Detection and Mitigation:

1. Certificate Transparency (CT):

CT logs provide publicly auditable records of certificates, making suspicious certificates easier to detect.

2. Certificate Revocation:

Fraudulent certificates should be revoked using mechanisms such as CRLs or OCSP, where applicable.

3. Secure CA Private Keys:

CA private keys should be protected using Hardware Security Modules (HSMs).

4. Offline Root CA:

Keep the root CA offline and use intermediate CAs for routine certificate issuance.

5. Monitoring:

Organizations should continuously monitor certificate logs for unexpected certificates for their domains.

6. Incident Response:

Once a CA breach is detected, affected certificates should be identified, revoked/replaced, and trust relationships reviewed.

Conclusion;

A CA breach can undermine the trust foundation of an entire PKI system. Certificate Transparency, certificate revocation, HSM-protected CA keys, offline root CAs, and continuous monitoring can reduce the impact and help detect fraudulent certificates.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.
Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

Answer:

Data integrity ensures that stored data has not been modified without authorization. Hashing can detect changes because even a small modification produces a different hash.

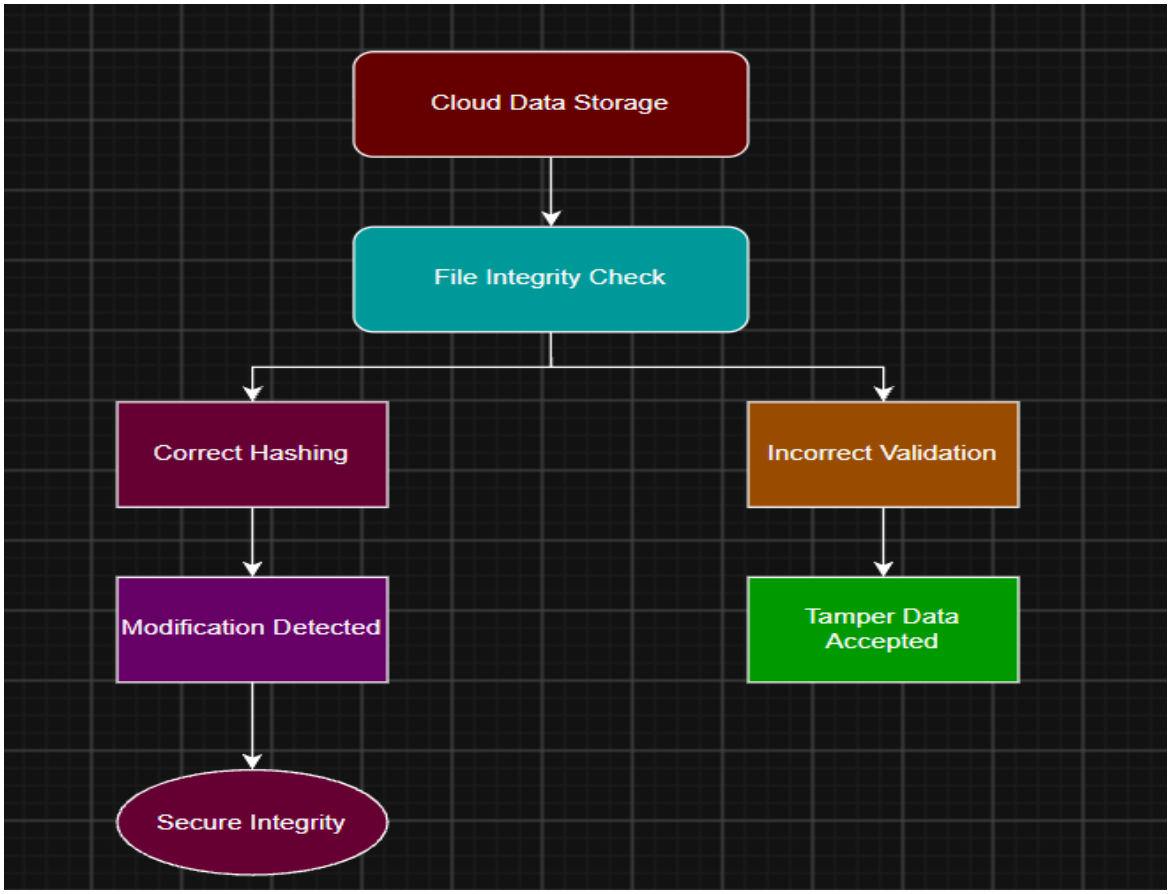
Original File → SHA-256 → Hash A

Modified File → SHA-256 → Hash B

Hash A ≠ Hash B
↓
Data modified

Analysis:

- The cloud provider uses hashing but has implemented the validation process incorrectly. Hashing itself may be strong, but incorrect verification can still result in integrity failures.
- For example, if the system does not securely store the original hash, does not recalculate the hash when retrieving the file, or compares the values incorrectly, a modified file may be accepted.
- Another important issue is that a plain hash does not provide authentication. If an attacker can modify both the file and its stored hash, simple hash comparison may not detect the attack.



Recommended Solution:

A stronger mechanism is HMAC-SHA-256, where a secret key is used along with the file.

File + Secret Key
↓
HMAC-SHA-256
↓
HMAC Value
↓
Store Securely

Retrieved File + Same Key
↓
Calculate HMAC
↓
Compare the values
↓
Match → Data valid
No Match → Tampering detected

Unlike a plain hash, an attacker who does not possess the secret key cannot simply calculate a valid HMAC for modified data.

Additional Measures:

- Use HMAC-SHA-256 for authenticated integrity.
- Protect the HMAC secret key using secure key management.
- Recalculate the HMAC whenever data is retrieved.
- Use constant-time comparison when comparing authentication tags.
- Maintain audit logs to identify suspicious modifications.
- For public verification or scenarios without a shared secret, digital signatures can be used instead.

Incorrect hash validation can allow modified cloud data to be accepted as genuine. HMAC-SHA-256 with secure key management and proper verification provides stronger integrity protection because attackers cannot generate a valid authentication code without the secret key.